

M365 — Automatisation PowerShell (Modules 2 et 3)

Formation : Administration Microsoft 365

Modules couverts :

- **Module 2** — Gestion des identités : utilisateurs, groupes, licences, Microsoft Graph
- **Module 3** — Sécurité : blocage de comptes, révocation de sessions, audit MFA

Durée indicative : 1 à 2 jours

Prérequis : TP PowerShell de base, tenant M365 de **laboratoire** ou de test, droits admin utilisateurs

Supports : [Syntaxe PowerShell](#) · [Cours M365 module 2](#) · [Module 3](#)

Fichiers de données fournis

Tous les CSV sont dans le dossier `data/`. Remplace `@contoso.com` par le domaine de ton tenant lab avant exécution.

FICHER	CONTENU	LIGNES
<code>nouveaux-utilisateurs.csv</code>	Arrivées — identité complète	8
<code>attribution-licences.csv</code>	SKU à assigner par UPN	8
<code>groupes-securite.csv</code>	Groupes à créer (Security / M365)	5
<code>membres-groupes.csv</code>	Appartenances groupe ↔ utilisateur	10
<code>onboarding-complet.csv</code>	Onboarding tout-en-un (capstone)	3
<code>departs.csv</code>	Départs — blocage / révocation	3
<code>utilisateurs-invalides.csv</code>	Données volontairement erronées (validation)	4

Colonnes principales

`nouveaux-utilisateurs.csv`

COLONNE	OBLIGATOIRE	EXEMPLE
<code>DisplayName</code>	✓	Jean Dupont
<code>UserPrincipalName</code>	✓	jean.dupont@contoso.com
<code>UsageLocation</code>	✓	FR
<code>TempPassword</code>	✓	Temp2026!Jd
<code>Department</code> , <code>JobTitle</code> , <code>ManagerUPN</code>		Métadonnées Entra

`attribution-licences.csv`

COLONNE	DESCRIPTION
<code>SkuPartNumber</code>	<code>ENTERPRISEPACK</code> (E3), <code>SPE_E3</code> , <code>ENTERPRISEPREMIUM</code> (E5)...
<code>DisabledPlans</code>	Plans à désactiver, séparés par <code>;</code> (vide = tout actif)

Consignes générales

RÈGLE	DÉTAIL
Tenant lab	Ne jamais exécuter sur un tenant de production sans validation
Mode test	Chaque script doit supporter <code>-WhatIf</code> ou <code>-DryRun</code> dès le niveau 4
Logs	Journaliser succès/échecs dans <code>logs/import-YYYYMMDD.log</code>
Secrets	Ne pas committer les mots de passe ; CSV fourni pour le lab uniquement
Modules	<code>Install-Module Microsoft.Graph -Scope CurrentUser</code>

Scopes Graph recommandés par étape

NIVEAU	SCOPES
4-5	User.ReadWrite.All , Organization.Read.All
6-7	+ Group.ReadWrite.All , Directory.ReadWrite.All
8-9	+ UserAuthenticationMethod.Read.All , AuditLog.Read.All (lecture MFA)
10	Combinaison des scopes ci-dessus

Connect-MgGraph -Scopes

"User.ReadWrite.All","Group.ReadWrite.All","Organization.Read.All"

Niveau 1 — Lire et explorer les CSV

Objectif : manipuler `Import-Csv` sans toucher au tenant.

Exercice 1.1 — Inventaire des arrivées

Importer `data/nouveaux-utilisateurs.csv` et afficher un tableau avec `DisplayName` , `Department` , `UserPrincipalName` .

Critère : 8 lignes affichées avec `Format-Table` .

Exercice 1.2 — Comptage par département

À partir du même CSV, grouper par `Department` et afficher le nombre d'utilisateurs par service (`Group-Object` + `Measure-Object`).

Critère : sortie du type `IT : 3` , `RH : 1` , etc.

Exercice 1.3 — Filtrer les licences E3

Importer `attribution-licences.csv`, filtrer les lignes où `SkuPartNumber` = `ENTERPRISEPACK`, exporter le résultat vers `output/e3-a-attribuer.csv`.

Critère : fichier CSV créé, sans ligne `#TYPE`.

Niveau 2 — Valider les données avant import

Objectif : construire une fonction de validation réutilisable (pas encore d'appel Graph).

Exercice 2.1 — Fonction Test-UpnValide

Créer `Test-UpnValide` qui retourne `$true` si l'UPN contient `@` et un domaine (regex simple : `^[^@]+@[^@]+\.[^@]+$`).

Tester sur les lignes de `nouveaux-utilisateurs.csv` et `utilisateurs-invalides.csv`.

Exercice 2.2 — Fonction Test-UsageLocation

Retourner `$false` si `UsageLocation` est vide ou différent de `FR` (pour ce lab).

Exercice 2.3 — Rapport de validation

Script `Validate-UsersCsv.ps1` qui :

1. Importe un CSV passé en paramètre `-CheminCsv`
2. Pour chaque ligne, vérifie UPN, `UsageLocation`, mot de passe ≥ 8 caractères
3. Exporte les lignes **invalides** vers `output/rejets-validation.csv` avec une colonne `Raison`
4. Affiche un résumé : `X valides / Y rejetées`

Critère : `utilisateurs-invalides.csv` → 4 rejets avec raisons explicites.

Niveau 3 — Connexion Graph et lecture du tenant

Objectif : premiers appels en **lecture seule** sur le tenant lab.

Exercice 3.1 — Connexion et contexte

Se connecter à Graph, afficher le tenant (`Get-MgContext`) : `TenantId` , `Account` , scopes accordés.

Exercice 3.2 — Inventaire des SKU

Lister les licences souscrites (`Get-MgSubscribedSku`) : `SkuPartNumber` , `ConsumedUnits` , unités disponibles. Exporter vers `output/sku-inventaire.csv` .

Critère : vérifier que les SKU du fichier `attribution-licences.csv` existent dans le tenant.

Exercice 3.3 — Croiser CSV et tenant

Script qui lit `nouveaux-utilisateurs.csv` et pour chaque UPN indique si l'utilisateur **existe déjà** (`Get-MgUser -Filter`) ou est à **créer**.

Exporter `output/statut-utilisateurs.csv` avec colonnes `UserPrincipalName` , `Statut` (`Existe` / `A créer`).

Niveau 4 — Création d'utilisateurs (simple → boucle)

Objectif : `New-MgUser` avec gestion d'erreurs progressive.

Exercice 4.1 — Créer un utilisateur unique

À partir de la **première ligne** de `nouveaux-utilisateurs.csv` , créer un utilisateur avec `New-MgUser -BodyParameter` :

- `MailNickname` = partie avant `@` de l'UPN
- `PasswordProfile.ForceChangePasswordNextSignIn = $true`
- `AccountEnabled = $true`

Critère : utilisateur visible dans Entra admin center.

Exercice 4.2 — Boucle avec journalisation

Script `Import-Users.ps1` :

```
param(
    [string]$CheminCsv = ".\data\nouveaux-utilisateurs.csv",
    [switch]$WhatIf
)
```

Pour chaque ligne valide :

- Si `-WhatIf` → afficher l'action sans créer
- Sinon → `New-MgUser` dans un `try/catch`
- Logger : `OK jean.dupont@contoso.com` ou `ERREUR ...`

Critère : relance sans doublon → erreur capturée, script continue.

Exercice 4.3 — Idempotence

Avant `New-MgUser`, vérifier si l'UPN existe. Si oui, passer à la ligne suivante avec message `SKIP`.

Critère : deuxième exécution = 8 skips, 0 erreurs.

Niveau 5 — Attribution des licences

Objectif : lier CSV licences aux utilisateurs créés.

Exercice 5.1 — Résoudre un SKU

Fonction `Get-SkuId` qui prend un `SkuPartNumber` et retourne le `SkuId` (GUID) via `Get-MgSubscribedSku`.

Exercice 5.2 — Assigner une licence

Pour une ligne de `attribution-licences.csv`, assigner la licence avec `Set-MgUserLicense` (tableau `AddLicenses` / `RemoveLicenses`).

Documentation : [Set-MgUserLicense](#)

Exercice 5.3 — Script complet licences

`Assign-Licenses.ps1` :

1. Importe `attribution-licences.csv`
2. Pour chaque UPN, vérifie que l'utilisateur existe
3. Assigne le SKU ; si `DisabledPlans` non vide, désactive les plans listés
4. Exporte `output/rapport-licences.csv` : `UPN`, `SkuPartNumber`, `Resultat`

Critère : utilisateur sans `UsageLocation` → erreur claire (rappel module 2).

Niveau 6 — Groupes et membres

Objectif : automatiser la structure groupe ↔ licence ↔ métier (module 2).

Exercice 6.1 — Créer un groupe de sécurité

À partir de la première ligne de `groupes-securite.csv` (type `Security`), créer le groupe avec `New-MgGroup` :

- `SecurityEnabled = $true`
- `MailEnabled = $false`

Exercice 6.2 — Créer un groupe Microsoft 365

Créer `TEAM-Projet-Migration` (`GroupType = M365`) :

- `MailEnabled = $true`

- `GroupTypes = @"Unified"`
-

Exercice 6.3 — Import groupes + membres

Script `Import-Groups.ps1` en deux phases :

Phase A — importe `groupes-securite.csv`, crée les groupes manquants (idempotent)

Phase B — importe `membres-groupes.csv`, ajoute chaque membre avec `New-MgGroupMember`

Gérer le cas « membre déjà présent » sans planter.

Critère : `Get-MgGroupMember` confirme les effectifs attendus.

Niveau 7 — Onboarding intégré (capstone intermédiaire)

Objectif : un script unique, données `onboarding-complet.csv`.

Exercice 7.1 — Script New-OnboardingUser

Fonction ou script paramétré qui pour **chaque ligne** :

1. Valide les champs obligatoires
2. Crée l'utilisateur (si absent)
3. Assigne la licence `SkuPartNumber`
4. Ajoute au groupe `GroupName`
5. Écrit une ligne dans `output/onboarding-rapport.csv`

Paramètres :

```
param(
    [Parameter(Mandatory)]
    [string]$CheminCsv,
    [switch]$WhatIf
)
```

Critère : 3 utilisateurs du CSV onboarding traités de bout en bout.

Niveau 8 — Sécurité et départs (Module 3)

Objectif : automatiser le **offboarding** et l'audit sécurité.

Exercice 8.1 — Bloquer un compte

À partir d'une ligne de `departs.csv` où `BloquerCompte = Oui` :

- `Update-MgUser -AccountEnabled:$false`
-

Exercice 8.2 — Révoquer les sessions

Si `RevoquerSessions = Oui` :

```
Revoke-MgUserSignInSession -UserId $userId
```

Script `Process-Departs.ps1` qui traite tout `departs.csv` avec log et `-WhatIf` .

Exercice 8.3 — Audit MFA (lecture)

Script `Export-MfaStatus.ps1` :

1. Liste les utilisateurs actifs (`Get-MgUser -Filter "accountEnabled eq true"`)
2. Pour chaque utilisateur, vérifie les méthodes d'authentification enregistrées (`Get-MgUserAuthenticationMethod`)
3. Exporte `output/mfa-status.csv` : `UPN` , `MfaEnregistre` (Oui/Non), `Methodes`

Scope : `UserAuthenticationMethod.Read.All`

Critère : identifier au moins les comptes sans MFA dans le tenant lab.

Exercice 8.4 — Rapport admins sans MFA

Croiser les utilisateurs ayant un rôle admin Entra (`Get-MgDirectoryRole` + `Get-MgDirectoryRoleMember`) avec le statut MFA. Exporter `output/admins-sans-mfa.csv` .

Lien cours : déploiement MFA prioritaire sur les administrateurs (module 3).

Niveau 9 — Projet final : moteur d'import complet

Objectif : assembler tout en un **orchestrateur** modulaire.

Architecture attendue

```
scripts/
├─ config.ps1           # domaine tenant, chemins, scopes
├─ functions/
│   ├─ Connect-Tenant.ps1
│   ├─ Import-UsersFromCsv.ps1
│   ├─ Assign-LicensesFromCsv.ps1
│   ├─ Import-GroupsFromCsv.ps1
│   ├─ Process-Offboarding.ps1
│   └─ Write-ImportLog.ps1
├─ Invoke-M365Provisioning.ps1  # point d'entrée
└─ logs/
```

Exercice 9.1 — Invoke-M365Provisioning.ps1

Point d'entrée avec paramètres :

```
param(
    [ValidateSet('Users','Licenses','Groups','Offboard','All')]
    [string]$Action = 'All',
    [string]$DataPath = '.\data',
    [switch]$WhatIf
)
```

ACTION	FICHIERS CSV UTILISÉS
Users	nouveaux-utilisateurs.csv
Licenses	attribution-licences.csv
Groups	groupes-securite.csv + membres-groupes.csv
Offboard	departs.csv
All	séquence complète dans l'ordre logique

Critères de réussite :

#	VÉRIFICATION
1	-WhatIf n'écrit rien dans le tenant
2	Chaque étape journalise dans logs/
3	Une erreur sur une ligne n'arrête pas tout le batch (try/catch par ligne)
4	Rapport final console : créés / ignorés / erreurs par catégorie
5	Exécution All sur tenant lab reproductible (idempotent)

Bonus 9.2 — Planification

Créer une tâche planifiée Windows ou un script `Register-ScheduledImport.ps1` qui lance l'import utilisateurs chaque nuit (documenter les prérequis : compte de service, secret, `Register-ScheduledTask`).

Grille d'évaluation

NIVEAU	EXERCICES	POINTS
1 — CSV	1.1 à 1.3	10
2 — Validation	2.1 à 2.3	15
3 — Lecture Graph	3.1 à 3.3	10
4 — Création users	4.1 à 4.3	15
5 — Licences	5.1 à 5.3	15
6 — Groupes	6.1 à 6.3	15
7 — Onboarding	7.1	10
8 — Sécurité	8.1 à 8.4	15
9 — Projet final	9.1	20
Bonus 9.2		+5
Total		125 (+5)

Rendu attendu

Dossier `TP-M365-Auto-{Nom}/` :

```
TP-M365-Auto-{Nom}/
├─ scripts/           # tous les .ps1
├─ output/           # CSV générés (sku, rapports, mfa...)
├─ logs/             # fichiers .log
└─ README.txt        # domaine tenant utilisé, scopes, commandes lancées
```

Rappels cours — mapping modules

NOTION COURS	EXERCICE
UPN, UsageLocation	2.x, 4.x
Licences par SKU / groupe	5.x, 6.x
Microsoft Graph PowerShell	3.x → 9.x
Moindre privilège (scopes)	tous
Bloquer compte / révoquer sessions	8.1, 8.2
MFA administrateurs	8.3, 8.4
Journal d'audit (export CSV)	8.3 (rapport MFA)

Formateur : Rayan Mechety — RedStone Formations